

Datové typy a proměnné

Datový typ

- **datový typ** určuje, jaký druh hodnoty může proměnná obsahovat
- určuje také:
 - velikost hodnoty v paměti
 - povolené operace
 - způsob reprezentace hodnoty
- příklady datových typů:
 - `int`
 - `float`
 - `double`
 - `bool`
 - `char`
 - `string`
 - `array`
 - `object`

Příklad:

```
int vek = 18;  
string jmeno = "Jan";  
bool jePrihlasen = true;
```

Proměnná

- **proměnná** = pojmenovaný prostor pro uložení hodnoty
- slouží k uložení hodnoty
- hodnota proměnné se může během programu měnit
- proměnná má:
 - název
 - datový typ
 - hodnotu
 - obor platnosti

Příklad:

```
int cislo = 10;
```

- `int` = datový typ

- `cislo` = název proměnné
- `10` = hodnota

Deklarace a inicializace

Deklarace

- **deklarace** = vytvoření proměnné
- říkáme tím programu:
 - jak se proměnná jmenuje
 - jaký má datový typ

Příklad:

```
int vek;
```

Inicializace

- **inicializace** = přiřazení první hodnoty proměnné
- proměnná začne obsahovat konkrétní hodnotu

Příklad:

```
vek = 18;
```

Deklarace a inicializace současně

```
int vek = 18;
```

Základní datové typy

Číselné typy

- slouží k ukládání čísel
- mohou být:
 - celočíselné
 - desetinné

Celočíselné typy

- ukládají celá čísla
- například:
 - `int`
 - `long`

- `short`
- `byte`

Příklad:

```
int pocet = 5;
```

Desetinné typy

- ukládají desetinná čísla
- například:
 - `float`
 - `double`
 - `decimal`

Příklad:

```
double cena = 99.90;
```

Boolean

- logický typ
- obsahuje hodnotu:
 - `true`
 - `false`
- používá se hlavně v podmínkách

Příklad:

```
bool jePlnolety = true;
```

Char

- jeden znak
- zapisuje se obvykle do apostrofů

Příklad:

```
char znak = 'A';
```

String

- textový řetězec

- skládá se ze znaků
- zapisuje se obvykle do uvozovek

Příklad:

```
string jmeno = "Jan";
```

Object

- obecný typ pro objekt
- může představovat složitější datovou strukturu
- v objektově orientovaném programování obsahuje:
 - atributy
 - metody

Statická a dynamická typovost

Staticky typované jazyky

- typ proměnné je známý při překladu programu
- proměnná má pevně daný typ
- do proměnné nelze později uložit hodnotu jiného typu
- chyby typů se často odhalí před spuštěním programu

Příklad v C#:

```
int x = 5;  
x = "ahoj"; // chyba
```

Příklady staticky typovaných jazyků:

- C#
- Java
- C
- C++

Poznámka:

- TypeScript přidává nad JavaScript statickou typovou kontrolu
- při spuštění se ale typy odstraní a běží JavaScript

Dynamicky typované jazyky

- typ se určuje až za běhu programu
- typ je vázaný spíše na hodnotu než na proměnnou

- do stejné proměnné lze uložit hodnoty různých typů
- chyby typů se často projeví až při spuštění

Příklad v JavaScriptu:

```
let x = 5;  
x = "ahoj"; // povoleno
```

Příklady dynamicky typovaných jazyků:

- JavaScript
- Python
- PHP

Poznámka:

- JavaScript má pro čísla typ `number`
- nerozlišuje klasicky `int`, `float`, `double`

Kompilované a interpretované jazyky

Kompilovaný jazyk

- zdrojový kód se před spuštěním přeloží
- výsledkem může být spustitelný soubor
- chyby se často odhalí už při překladu

Příklady:

- C
- C++
- C#

Interpretovaný jazyk

- kód se obvykle vykonává postupně za běhu
- program spouští interpret
- chyby se mohou projevit až při běhu programu

Příklady:

- Python
- JavaScript

Poznámka:

- v praxi to není vždy úplně černobílé
- některé jazyky používají kombinaci překladu a interpretace

Hodnotové a referenční typy

Hodnotové typy

- proměnná obsahuje přímo hodnotu
- při přiřazení se obvykle kopíruje hodnota
- změna kopie neovlivní původní proměnnou

Příklad:

```
int a = 5;
int b = a;
b = 10;

// a je stále 5
```

Typické hodnotové typy:

- `int`
- `float`
- `double`
- `bool`
- `char`

Referenční typy

- proměnná neobsahuje přímo objekt
- obsahuje referenci na objekt v paměti
- při přiřazení se kopíruje reference
- více proměnných může odkazovat na stejný objekt
- změna objektu přes jednu proměnnou se projeví i přes druhou

Příklad:

```
Person a = new Person();
Person b = a;

b.Name = "Jan";

// a.Name je také "Jan"
```

Typické referenční typy:

- `object`
- `string`
- `array`
- `class`
- `interface`

Poznámka:

- `string` je referenční typ, ale chová se jako immutable
- jeho obsah se po vytvoření nemění

Reference

- **reference** = odkaz na objekt nebo hodnotu v paměti
- neobsahuje přímo data
- obsahuje informaci, kde se data nacházejí
- používá se hlavně u objektů a složených struktur

Příklad zjednodušeně:

proměnná osoba → objekt v paměti

Konstanta a imutabilita

Konstanta

- **konstanta** = pojmenovaná hodnota, které po nastavení nelze přiřadit novou hodnotu
- hodnota je pevně daná
- používá se pro hodnoty, které se nemají měnit

Příklad v JavaScriptu:

```
const x = 7;  
x = 4; // chyba
```

Příklad v C#:

```
const double PI = 3.14159;
```

Imutabilita

- **imutabilita** znamená, že objekt nebo hodnota se po vytvoření nemění
- místo změny původního objektu se vytvoří nový objekt
- používá se kvůli:
 - bezpečnosti

- předvídatelnosti
- jednodušší práci se stavem

Příklad:

```
const user = { name: "Jan", age: 18 };

// místo změny user.age se vytvoří nový objekt
const newUser = { ...user, age: 19 };
```

Rozdíl:

- konstanta brání změně proměnné
- imutabilita brání změně obsahu hodnoty nebo objektu

Poznámka:

- v JavaScriptu `const` neznamena automaticky immutable objekt
- znamená, že proměnná nemůže odkazovat na jiný objekt

Příklad:

```
const user = { name: "Jan" };
user.name = "Petr"; // povoleno

user = {}; // chyba
```

Obor platnosti proměnné

- **obor platnosti** určuje, kde je proměnná dostupná
- anglicky **scope**
- proměnná existuje jen v určité části programu

Lokální proměnná

- existuje uvnitř funkce, metody nebo bloku
- mimo tento blok není dostupná

Příklad:

```
void Test()
{
    int x = 5;
}
```

- `x` je dostupné jen uvnitř metody `Test`

Globální proměnná

- je dostupná ve větší části programu
- někdy v celém souboru nebo aplikaci
- může zhoršit přehlednost programu
- je vhodné ji používat opatrně

Blokový scope

- proměnná existuje jen uvnitř bloku
- blok je obvykle ohraničený složenými závorkami

Příklad:

```
if (true)
{
    int x = 10;
}
```

- `x` je dostupné jen uvnitř bloku `if`

Přetypování

- **přetypování** = změna hodnoty z jednoho datového typu na jiný
- používá se, když potřebujeme pracovat s hodnotou jako s jiným typem

Implicitní přetypování

- provede ho jazyk automaticky
- používá se tam, kde nehrozí ztráta dat
- typicky z menšího typu na větší

Příklad:

```
int a = 10;
double b = a;
```

- `int` se automaticky převede na `double`

Explicitní přetypování

- musí ho zapsat programátor
- používá se tam, kde může dojít ke ztrátě dat
- programátor tím říká, že s převodem počítá

Příklad:

```
double x = 10.8;  
int y = (int)x;
```

- výsledek bude 10
- desetinná část se ztratí

Konverze

- složitější převod mezi typy
- často se používají metody
- typicky u převodu mezi textem a číslem

Příklad:

```
string text = "123";  
int cislo = int.Parse(text);
```

Další příklady:

```
Convert.ToDouble("12.5");  
DateTime.Parse("2026-05-10");
```